
Reliability & Fallback Design Patterns for Multi-Agent LLM Systems

Muhammad Ashar Ishfaq
The Islamia University of Bahawalpur

Muhammad Asad Ishfaq
Indiana University Bloomington

Abstract

We present an empirical side-by-side comparison of orchestration-level fallback strategies—not new algorithms—on a fixed multi-agent QA pipeline. On 150 HotpotQA questions $\times$ three trials (450 logged runs per condition), we compare two baselines (no fallback, blind retry) against four targeted recovery patterns and one tested composition (P1+P2), reporting pass rate, p95 latency, and cost together. Targeted fallbacks that address *common recoverable failure modes* in our Claude Sonnet testbed (especially schema/JSON breakage) significantly beat no fallback under question-level McNemar tests with Bonferroni correction (P1/P2/P3/P5 vs. B0: each $p < 10^{-6}$). Blind retry does not ($p=0.0265$, n.s. after correction). P4 also beats B0 ($p=7.2 \times 10^{-5}$) but is worse than P3 on pass rate at $2.5\times$ the cost ($p=5.1 \times 10^{-4}$ for P3 vs. P4). The tested P1+P2 composition does not beat the best single targeted pattern ($p=0.37$ vs. P1). A GPT-4o-mini probe on the same question IDs shows no significant lift when structural failures are rare—so gains are conditional on the baseline error profile, not universal. We release code, data, and logs.

1 Introduction

Multi-agent LLM pipelines fail mid-run: bad sub-questions, empty retrieval, broken JSON, or ungrounded citations. Agent frameworks and production stacks commonly expose *retry loops* or halt on error—Reflexion-style verbal retry [Shinn et al., 2023], self-consistency sampling [Wang et al., 2023], and targeted re-roll after diagnosis [Zhu et al., 2025]—but rarely compare orchestration policies on *end-to-end* failure, latency, and cost under identical workloads.

Contribution. We do not propose new fallback algorithms. We benchmark seven fixed conditions on the same 150 HotpotQA questions [Yang et al., 2018] and corpus in a Planner–Retriever–Synthesizer–Formatter LangGraph pipeline: **baselines** B0 (no fallback) and B1 (blind retry), **vs. four targeted patterns** P1–P4 (tool-grounded retry, checkpointing, degradation, cross-agent verification) and one **tested composition** P5 (P1 then P2). Prior work validates individual techniques (CRITIC-style tool critique [Gou et al., 2024], cascades [Chen et al., 2024], debate [Du et al., 2024]); what was missing is a multi-objective menu under one testbed—including whether stacking two targeted patterns beats either alone.

Headline findings (Claude Sonnet, this testbed). Targeted fallbacks beat B0 when they match prevalent recoverable failures (here, largely `broken_json`; McNemar on 150 paired questions, Section 5). Blind retry does not ($p=0.0265$ vs. B0, n.s. after Bonferroni) and stretches p95 latency. P4 beats B0 ($p=7.2 \times 10^{-5}$) but trails P3 on pass rate at higher cost. P5 (the only composition we test) does not add significant gain over P1 ($p=0.371$). On GPT-4o-mini, B0/P1/P3 are 69%/68%/71% (n.s.)—gains are *failure-mode-dependent*, not universal.

Section 2 situates prior work. Sections 3–4 describe the pipeline and statistics. Section 5 reports pooled metrics with question-level significance tests. Appendix A holds ablations, failure types, 2WikiMultihopQA, and the GPT-4o-mini table.

2 Related Work

Much LLM reliability work is try-again: Self-Consistency [Wang et al., 2023] votes over samples; Reflexion [Shinn et al., 2023] retries with verbal memory; CRITIC [Gou et al., 2024] revises using tool signals. Intrinsic self-correction without external feedback often fails [Huang et al., 2024, Kamoi et al., 2024], motivating our tool/schema checks. FrugalGPT [Chen et al., 2024] budgets cost and quality via cascades but not mid-graph orchestration fallbacks. Multiagent Debate [Du et al., 2024] and LLM-as-judge [Zheng et al., 2023] sit in the same tradeoff space as P4. Production RCA [Tan et al., 2026, Roy et al., 2024] diagnoses deployed systems after the fact. Agent surveys [Wang et al., 2024, Song et al., 2026] map failure modes; AgentDebug [Zhu et al., 2025] shows targeted re-roll can beat undifferentiated retry for one mechanism. Few works report failure rate, p95 latency, and dollar cost on the same runs for several orchestration fallbacks on one multi-agent testbed.

3 Method

3.1 Testbed

Four stages: **Planner**, **Retriever**, **Synthesizer**, **Formatter**. Retriever searches a fixed local Hotpot-derived corpus (no live web). LLM stages use Claude Sonnet (claude-sonnet-4-6) at temperature 0, except P3’s rare Haiku path. Typed shared state exposes stage failures to fallbacks. Success is a scored-correct final answer (Section 4).

3.2 Conditions

Seven conditions represent structurally distinct recovery mechanisms—not an exhaustive design-space search; self-consistency and debate are related but distinct (Section 2).

B0 — No fallback. One pass; hard error or wrong final answer counts as failure.

B1 — Blind retry. Re-run failing stage/graph up to a cap with the same prompts/tools; no new evidence or rollback.

P1 — Tool-grounded retry. External checks (retrieval, citation IDs, JSON schema); capped stage re-run with check feedback.

P2 — Deterministic checkpointing. Restore last trusted checkpoint and re-execute forward after failure.

P3 — Graceful degradation. Simplify path (e.g., skip planning, Haiku cascade) after repeated failure or budget exhaustion.

P4 — Cross-agent verification gate. Second Verifier agent before formatting; one capped Synthesizer re-run on reject.

P5 — Composed P1+P2. P1 tool-grounded retry first; on continued failure, P2 checkpoint rollback (cap 2 retries/stage). Most questions recover within P1’s capped checks and never enter the checkpoint branch; when they do, rollback avoids stacking the long blind-retry chains that inflate B1’s tail, which helps explain P5’s lower p95 than P1 despite extra orchestration logic.

P1–P3 and P5 use external signals rather than intrinsic self-correction [Huang et al., 2024, Kamoi et al., 2024]; P4 is an LLM judge.

Table 1: Pooled results across three trials (450 runs per condition). Pass% pools all runs; inferential tests use 150 paired questions (Table 2).

Cond	Runs	Pass%	p50 (ms)	p95 (ms)	\$/q
B0	450	55.3	6440	11180	0.0052
B1	450	61.1	6889	30793	0.0085
P1	450	76.0	7951	20528	0.0072
P2	450	76.2	8047	18444	0.0073
P3	450	78.2	7764	18444	0.0073
P4	450	68.2	12366	33553	0.0183
P5	450	74.7	7321	12338	0.0071

Table 2: Selected pairwise tests on 150 Hotpot questions (majority pass per question; McNemar; Bonferroni $\alpha=0.002381$).

Comparison	Δpp	p	Cohen’s h	sig
P1 vs B0	+21.3	1.06×10^{-7}	0.45	**
P2 vs B0	+20.7	8.14×10^{-7}	0.44	**
P3 vs B0	+24.0	5.43×10^{-9}	0.52	**
P5 vs B0	+19.3	4.93×10^{-7}	0.41	**
P4 vs B0	+14.7	7.23×10^{-5}	0.30	**
B1 vs B0	+6.0	0.0265	0.12	ns
P3 vs P1	+2.7	0.134	—	ns
P5 vs P1	−2.0	0.371	—	ns
P4 vs P3	−9.3	5.12×10^{-4}	—	**

4 Experimental Setup

Dataset. 150 HotpotQA distractor validation questions [Yang et al., 2018] (short gold; bridge-preferred filters). Corpus: 300 supporting + 20 distractor documents; retriever searches only this set.

Scoring. Normalized token-coverage against gold (exact, substring, or gold-token coverage). We do not run a separate semantic-equivalence or factuality judge on the main metric; paraphrases can fail and citation quality is not scored directly (Section 7). All conditions share the same scorer.

Trials. Three independent trials on the same 150 questions (450 logged runs per condition). Pass rates, Wilson 95% CIs, latency, and cost in Table 1 and Figures 1–3 pool all 450 runs per condition.

Inferential testing. Because the three trials reuse the same 150 questions, run-level outcomes are not independent. All primary pass-rate significance tests are therefore performed at the question level: for each condition and question, we label pass if a *majority* of that question’s trials pass, yielding 150 paired binary outcomes per condition pair. We apply two-sided McNemar’s test (with continuity correction) and Bonferroni correction over 21 condition pairs ($\alpha=0.05/21=0.002381$). Table 2 reports Δpp and Cohen’s h on these question-level majority rates.

Metrics. Pass rate, end-to-end latency (p50/p95), average cost per question.

5 Results

5.1 Headline

Table 1 summarizes pooled runs (450 per condition). Table 2 lists McNemar tests on 150 paired questions. P1, P2, P3, and P5 each beat B0 with $p < 10^{-6}$. B1 does not ($p=0.0265$, n.s. after Bonferroni). P4 beats B0 ($p=7.2 \times 10^{-5}$) but loses to P3 ($p=5.1 \times 10^{-4}$) at $2.5\times$ the cost.

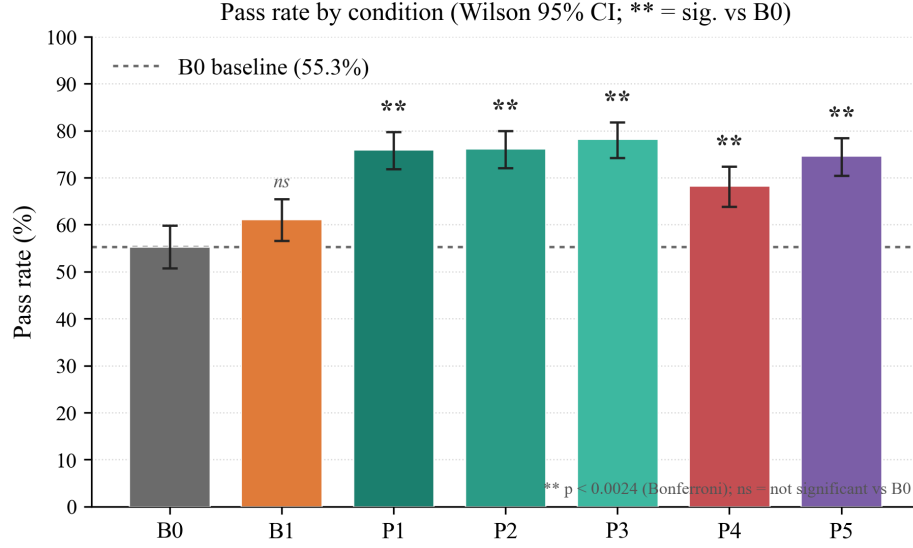


Figure 1: Pooled pass rate with Wilson 95% CIs. Markers vs. B0: McNemar on 150 paired questions, Bonferroni $\alpha=0.002381$.

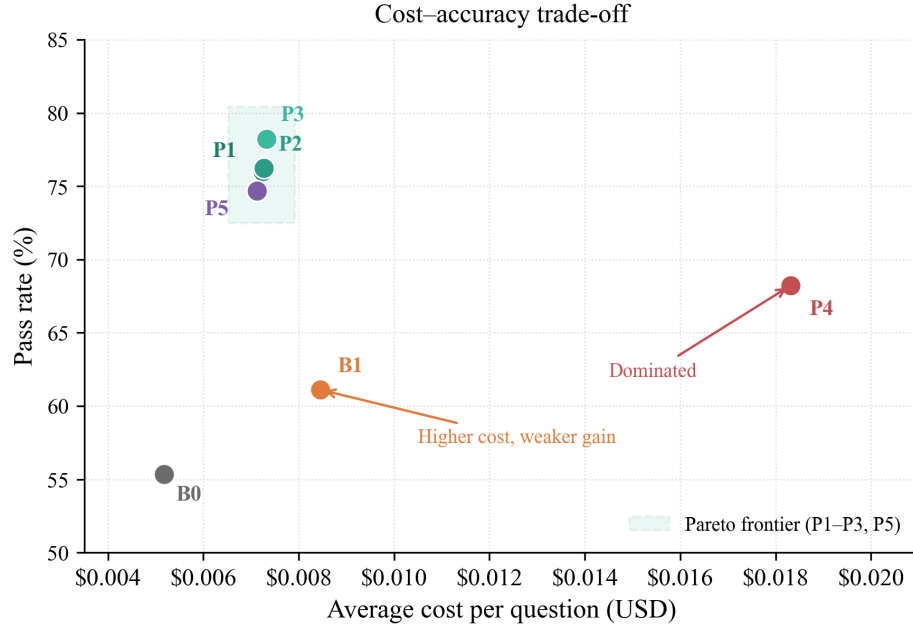


Figure 2: Cost vs. accuracy. Dashed box: P1–P3/P5 cluster.

5.2 Pass rate, cost, and latency

Figure 1 shows pooled pass rates with Wilson 95% CIs (450 runs). B0 55.3%; P1–P3/P5 74.7%–78.2%; B1 61.1%; P4 68.2%. P5 is not significantly above P1 ($p=0.371$): we find *no additional benefit from the tested P1+P2 composition* (we did not evaluate other compositions).

Figure 2 (cost vs. accuracy) and Figure 3 (p95 latency) show the P1–P3/P5 cluster near \$0.007/q with strong pass rates, while B1 and P4 pay more for weaker accuracy and longer tails.

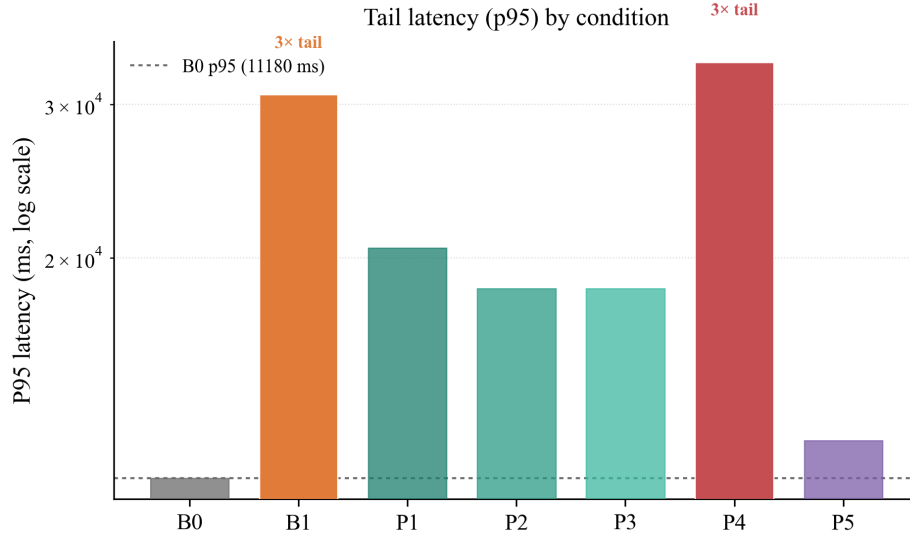


Figure 3: Pooled p95 latency (log scale). B1 and P4 stretch the tail vs. B0.

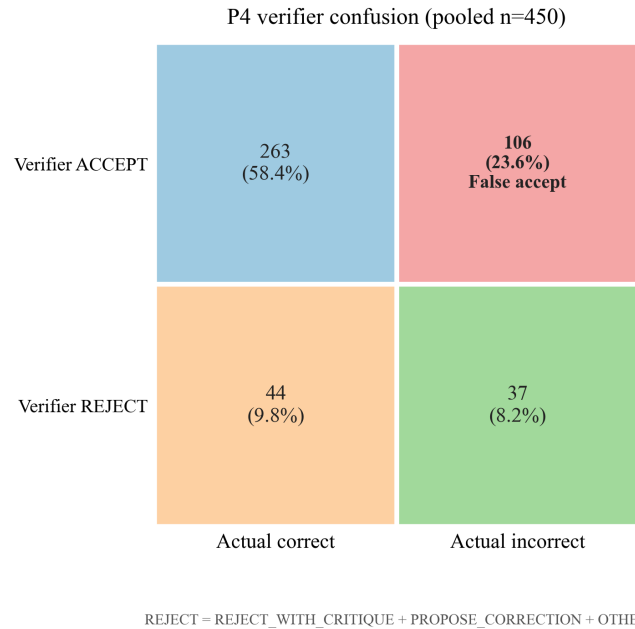


Figure 4: P4 verifier confusion (450 pooled runs). False accepts: 23.6%.

5.3 Composition, stages, and P4

P5 does not significantly outperform P1/P2/P3 ($p > 0.13$ among targeted cluster). P1–P3 shift failures from Synthesizer to Formatter (Appendix A, Figure 6). P4’s verifier false-accept rate is 23.6% (Figure 4), explaining weak cost–reliability tradeoffs despite significance vs. B0.

6 Discussion

Choose fallbacks to match *observed* failure modes. On Claude Sonnet here, schema/JSON checks recover large structural mass; on GPT-4o-mini (Appendix), the same checks barely move pass rate when that mass is absent. Do not default to blind retry (n.s. vs. B0 after correction; heavy p95). Do

not assume a verifier gate is automatically safer: P4 is significant vs. B0 ($p=7.2 \times 10^{-5}$) but has lower pass rate and higher cost than P3 ($p=5.1 \times 10^{-4}$ for P3 vs. P4).

Tool/schema checks with capped retry (P1), checkpoint rollback (P2), degrade (P3), or the tested P1-then-P2 composition (P5) cluster near \$0.007/q with much better pass rates than B0. Reliability looks like orchestration hygiene—external checks, checkpoints, degrade paths—not “add another agent.”

Reproducibility. Code, analysis scripts, and logs: <https://github.com/Ashar086/llm-fallback-reliability>.

7 Limitations

Scope. One primary model family (Claude Sonnet; Haiku on P3’s rare path), one main benchmark (150 unique HotpotQA questions), one pipeline topology. Gains on Claude are strongly driven by structural failures (broken_json); GPT-4o-mini weakens generalization (Appendix).

Statistics. We report pooled 450-run descriptives but test significance on 150 paired questions; trials are not independent.

Scoring. Token-coverage gold matching is brittle; automated semantic equivalence or factuality scoring is future work.

Composition. Only P1+P2 was composed; other stacks may differ.

Failure taxonomy. Covers structural/content failures in this testbed, not infrastructure failures (time-outs, rate limits, API outages, vector-DB failures).

Acknowledgments and Disclosure of Funding

This work received no external funding.

References

- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research (TMLR)*, 2024.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, 2024.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024.
- Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics (TACL)*, 2024. URL <https://aclanthology.org/2024.tacl-1.78/>.
- Devjeet Roy, Xuchao Zhang, Rashi Bhawe, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. Exploring LLM-based agents for root cause analysis. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024. URL <https://www.microsoft.com/en-us/research/publication/exploring-llm-based-agents-for-root-cause-analysis/>. Industry Track.

Table 3: P1 check ablation ($n=100$ runs). Wilson 95% CIs; z -tests vs. full P1 and B0 on same IDs.

Variant	Pass% [Wilson 95%]	n	vs. full P1	vs. B0
B0	62.0 [52.2, 70.9]	100	−17.0 pp, $p=0.0084$	—
P1-full	79.0 [70.0, 85.8]	100	—	+17.0 pp, $p=0.0084$
P1R (retrieval-only)	61.0 [51.2, 70.0]	100	−18.0 pp, $p=0.0055$	−1.0 pp, $p=0.8845$
P1C (citation-only)	57.0 [47.2, 66.3]	100	−22.0 pp, $p=0.0009$	−5.0 pp, $p=0.4714$
P1S (schema-only)	74.0 [64.6, 81.6]	100	−5.0 pp, $p=0.4044$	+12.0 pp, $p=0.0689$

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Peiyang Song, Pengrui Han, and Noah Goodman. Large language model reasoning failures. *Transactions on Machine Learning Research (TMLR)*, 2026.

Gou Tan, Zilong He, Min Li, Haiyu Huang, Yilun Wang, Pengfei Chen, Giuliano Casale, and Chuanfu Zhang. LLMRCA: Multilevel root cause analysis for LLM applications using multimodal observability data. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2026. doi: 10.1145/3806200.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Datasets and Benchmarks Track.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025. AgentDebug.

A Additional analyses

A.1 P1 check ablation

P1 combines retrieval, citation, and schema checks. One-check ablations on 50 Hotpot questions $\times$ two trials ($n=100$ runs per variant):

Schema-only carries most single-check lift; retrieval/citation alone do not beat B0.

A.2 Terminal failure types

B0: 154 broken_json (34.2%) vs. 47 wrong_answer (10.4%). P1–P3 collapse broken_json to 0–6 runs; wrong_answer rises to 21.8–22.7% as runs complete and are scored.

Table 4: Terminal failure types (count and % of 450 runs per condition).

Failure type	B0	B1	P1	P2	P3	P4	P5
broken_json	154 (34.2%)	130 (28.9%)	6 (1.3%)	7 (1.6%)	0 (0.0%)	10 (2.2%)	5 (1.1%)
wrong_answer	47 (10.4%)	45 (10.0%)	102 (22.7%)	100 (22.2%)	98 (21.8%)	133 (29.6%)	109 (24.2%)
Total fails	201 (44.7%)	175 (38.9%)	108 (24.0%)	107 (23.8%)	98 (21.8%)	143 (31.8%)	114 (25.3%)

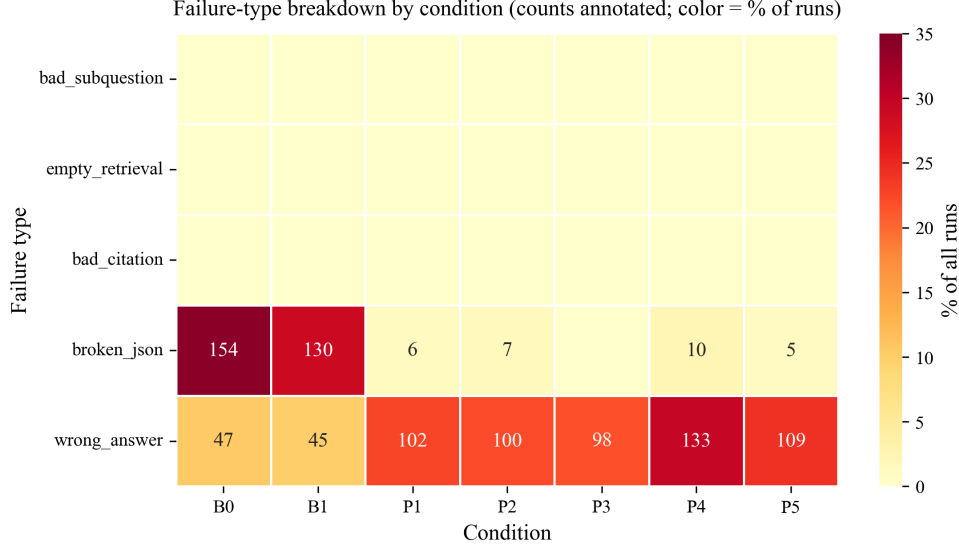


Figure 5: Terminal failure-type heatmap (% of runs).

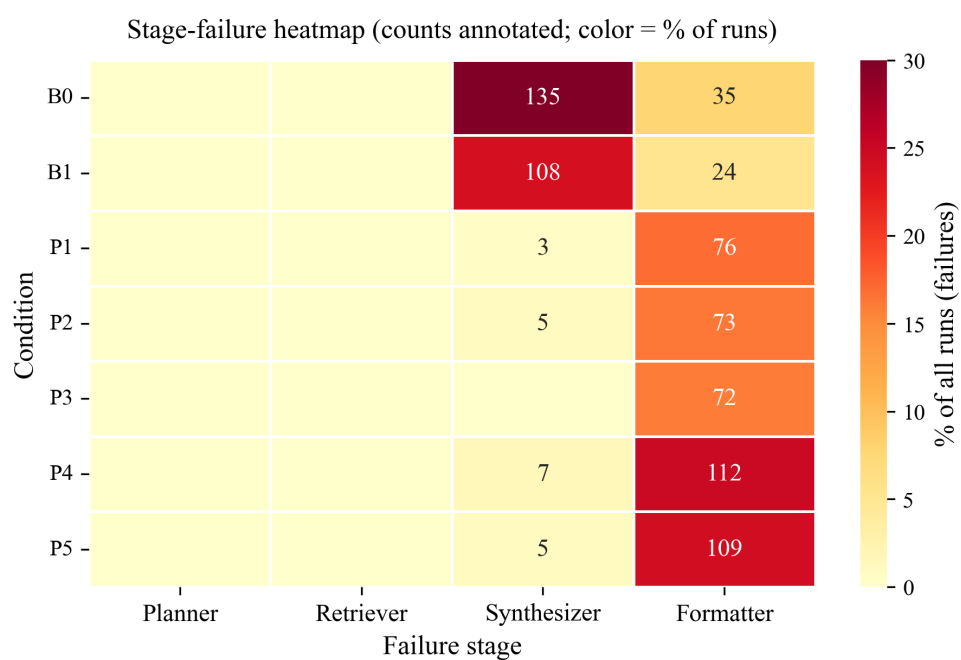
A.3 Stage-failure heatmap

A.4 2WikiMultihopQA validation

B0/B1/P1/P3 on 50 2Wiki questions ($n=100$ runs/condition): ranking holds (P3 +30 pp vs. B0, $p \approx 1.6 \times 10^{-5}$; P1 +18 pp, $p=0.008$; B1 n.s.).

A.5 GPT-4o-mini probe

Same 50 Hotpot IDs ($n=100$ /condition): B0 69% (69/100; Wilson CI [59.4%, 77.2%]), P1 68% (−1.0 pp vs. B0, $p=0.879$), P3 71% (+2.0 pp, $p=0.758$). No significant lift over B0; baseline *broken_json* is rare, so tool checks have little to repair.



P1–P3 shift failures from Synthesizer to Formatter, indicating tool-grounded checks catch draft errors but formatting remains a residual failure mode.

Figure 6: Stage-failure heatmap. P1–P3 shift mass from Synthesizer to Formatter.